

CTO ONBOARDING · DOCUMENT 2 / 2

한대장 코드·아키텍처 상세

빌드 없는 정적 PWA + GitHub Actions 데이터 파이프라인 + 엣지 워커 2기

스택 Vanilla JS(모듈 번들러 없음) · GitHub Actions(Node 20, Playwright) · Cloudflare Workers/KV/Pages

코드 약 26,000줄 (앱 7.5k · 수집기 6.7k · 검증 5.2k · 관리자·도구 3.8k · 워크플로 2.1k · 서버 0.7k) · 기준 2026-08-20 main HEAD · 커밋 646

01 시스템 지형도

서버가 사실상 없다. 앱은 GitHub Pages에 올라간 **정적 파일**이고, 데이터를 만드는 일은 전부 **GitHub Actions** 위의 **로봇**이 한다. 로봇의 산출물은 저장소의 JSON 파일이며, 앱은 그 JSON을 fetch할 뿐이다. "백엔드 = 저장소"라는 구조가 이 프로젝트의 모든 설계 결정을 지배한다.

계층	실행 위치	구성 요소와 역할
클라이언트	브라우저 (PWA · 설치형)	index.html/app.js/style.css 화면 · match-engine.js 자격 판정 · forms.js + form-plan.js 신청서 · notify.js + notify-rules.js 알림 · chat.js 도우미 · sw.js 캐시·백그라운드
데이터	저장소의 정적 JSON (동일 오리진)	data/registered.json (등록 188) · notices.json (피드 615) · forms.json (양식 48) · majors.json (학과 14,312) · search-index.json (159)
파이프라인	GitHub Actions (예약·push 트리거)	워크플로 19개 — 수집 2종 · 원문 심층수집 · 링크 사냥/복구/정찰 · 자동 등록 · 양식 스키마화 · 누락 감사 · 배포 동기화 · 잠금 감시 · 진척도 갱신
엣지 서버	Cloudflare Workers (무료 등급)	server/push/ 푸시 발송(가동 중, KV) · server/chat/ 도우미 AI 선택기(미배포) · server/mail-worker.js 접수 메일(키 대기)
운영 콘솔	Cloudflare Pages + Access	_admin/ 6개 화면 — 저장소를 직접 쓰지 않고 admin-apply.yml 을 깨워 감사 통과 시에만 반영

이 코드베이스의 제1 설계 원칙 — "판정 규칙은 한 파일에만 둔다"

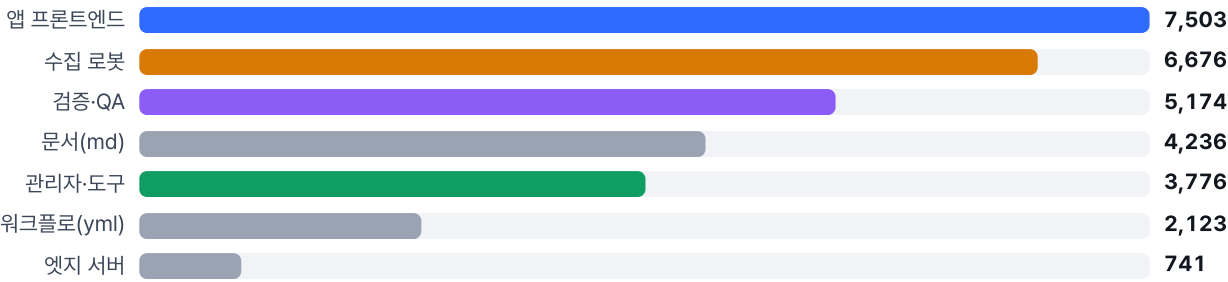
같은 판단을 두 곳에 적으면 **화면과 알림이 서로 다른 말을 한다**. 실제로 겪은 사고라서, 판정 로직은 브라우저·서비스워커·Node에서 **같은 파일**을 공유하도록 분리돼 있다.

공유 모듈	쓰는 곳	베끼면 생기는 일
match-engine.js	화면 · 서비스워커(백그라운드 알림) · 감사 스크립트	화면에서 숨긴 공고를 알림이 알린다
notify-rules.js	화면 알림 · 서비스워커 푸시 수신	같은 알림이 두 번 가거나 규칙이 갈라진다
form-plan.js	신청서 화면 · 감사 · 관리자 화면	"화면은 12문항인데 감사는 20문항"이 된다
collector/canon-url.mjs url-key.mjs · clean-title.mjs	수집기 · 자동 등록 · 관리자 쓰기 · 협업 병합기	같은 공고가 두 번 등록된다(중복 판정이 어긋남)

브라우저 전역 스크립트와 Node module.exports 를 겸용하는 형태로 작성돼 있다(번들러가 없으므로).

02 저장소 지도와 코드 규모

영역별 코드 규모 (줄 수)



검증 코드가 앱 코드의 69%다. 사람이 코드를 못 읽는 창업자 1인 + AI 세션이라는 개발 형태에서, 회귀를 잡아 주는 유일한 장치가 이 드라이버들이라 의도적으로 두 겹다(→ 09장).

경로	내용
/ (루트)	앱 본체 13개 파일 — 빌드 산출물이 아니라 브라우저가 그대로 읽는 원본
data/	앱이 fetch하는 JSON. 로봇과 사람의 공동 산출물이자 사실상의 DB
collector/	수집·정규화·자동등록·스키마화 모듈 30여 개 + 로봇의 기록장(<code>seen.json</code> , <code>health.json</code> , 리포트)
collector/extracted/	공고 원문·첨부 원본·HWP 추출 텍스트 415개 — 발췌·스키마화의 근거 자료(약 91MB)
verify/	Playwright 회귀 드라이버 + 감사/리포트 도구 24개
_admin/ · tools/	운영 콘솔 화면과 그 쓰기 경로
server/	Cloudflare Worker 3종(푸시·챗 선택기·메일)
CLAUDE.md (2,221줄)	세션 인수인계 문서 — 이 저장소의 실질적 설계 문서. 원칙·기술 사실·"되돌리면 안 되는 지점"이 누적돼 있다

03 프론트엔드 런타임

파일	줄	책임과 주의점
app.js	1,737	화면 렌더·온보딩·서류 보관함·신청 플로우·진척도 트래커·데이터 로더. 전역 <code>state = { profile, applications }</code> 하나를 <code>localStorage['handaejang.v1']</code> 에 저장. 학교 개편·분교 분리에 대비한 프로필 마이그레이션 (<code>migrateBranchCampus</code>) 이 로드 시 돈다 — 없으면 분교 학생이 본교 공고를 받는 다
match-engine.js	314	자격 판정·적합도·오래된 공고 숨김·실시간 공고 소유 판정(→ 04장)
forms.js	540	스키마 → 문서(<code>renderFormDoc</code>) 생성. .doc 저장·인쇄·공유
form-plan.js	299	스키마를 건드리지 않고 화면에 낼 질문만 재설계(→ 05장)
notify.js / notify-rules.js	744 / 336	알림 UI·권한 요청·알림함 / 규칙 엔진(순수 로직)
chat.js / chat-config.js	1,054 / 60	장학금 도우미. 판정을 새로 만들지 않고 <code>evaluate·dday</code> 를 그대로 호출(→ 08장)
data.js	425	대학 212·별칭 110·계열 8·캠퍼스·서류슬롯 8·제출채널 7·상시 제도 7 등 정적 카탈로그
sw.js	278	캐시 전략 + 백그라운드 확인 + 푸시 수신

■ 저장 위치 — 개인정보는 기기 밖으로 나가지 않는다

데이터	저장소	비고
프로필(학교·성적·소득구간·특별자격·연락처·계좌·주민번호)	localStorage	외부 전송 코드 없음. 주민번호는 화면에서 뒷자리 마스킹
증명서류 파일 8슬롯	IndexedDB (Blob)	사용자가 직접 공유/메일을 열 때만 기기 밖으로 나간다
신청서 답변·진척도·알림 발송 장부	localStorage	알림 중복 방지 장부는 120일 TTL
푸시 구독	Cloudflare KV	폰 주소 + 학교 + 캠퍼스만 저장 (→ 07장)

■ 서비스워커 캐시 전략 (sw.js, CACHE = handaejang-v58)

자원	전략	이유
앱 코드(html/js/css)	네트워크 우선 + 3.5초 타임아웃 폴백	예전엔 캐시 우선이라 버전 올리는 걸 잊으면 설치된 앱에 영영 반영되지 않았다 . 사람 기억에 의존하던 구멍을 막은 것
데이터(data/* .json)	네트워크 우선 (<code>cache: 'no-cache'</code> 강제 재검증)	Pages의 10분 캐시 때문에 배포가 늦게 보이던 문제 해결. 공고·양식 추가는 앱 코드 변경 없이 자동 반영
아이콘·이미지	캐시 우선	거의 안 바뀐다

버전 규칙(사고 이력): 두 개발자가 각각 같은 번호로 캐시 버전을 올려 "내용이 다른 같은 버전"이 두 번 생겼다. 합칠 때는 **양쪽보다 큰 번호**로 올려야 옛 캐시가 청소된다.

04 매칭 엔진

`evaluate(sch, profile)` 는 공고의 `eligibility` 객체와 프로필을 대조해 **상태·사유 목록·부족 정보** 세 가지를 돌려준다. 사유를 함께 돌려주는 것이 핵심이다 — "왜 안 되는지"를 화면에 그대로 보여 주기 때문에 사용자가 앱을 의심하지 않는다.

판정 신호 (12)

키	의미
minGpa	최소 평점 (신입생은 면제)
maxBracket	학자금 지원구간 상한
years	학년 제한
freshmanOnly	신입학 첫 학기
tracks	전공 계열
flagsAny	특별자격 5종 중 하나 이상
seoulOnly	거주지
needCert	공인 여학생적
exchange	교환학생 파견 예정
schoolOnly	재학 대학 한정
campusOnly	캠퍼스 한정
selective	선발 심사형(자격≠선정)

4단계 상태

- **eligible** — 요건 전부 충족, 바로 신청
- **selective** — 자격은 되나 선발 심사
- **unknown** — 프로필에 빠진 값이 있어 판단 불가 (`missing`에 무엇이 필요한지 담긴다)
- **ineligible** — 요건 미충족 (`reasons`에 사유)

적합도 점수 `fitScore`

기본 62점 → 충족 조건 수 × 3(최대 +15) → selective -8 / unknown -22 → 평점 여유 ($\text{gpa} - \text{minGpa}$) × 10 (최대 +12) → 소득구간 여유 (최대 +6) → 특별자격 매칭 +6 → 5~99로 클램프. **정렬용 휴리스틱**이지 확률이 아니다.

함수

하는 일 / 왜 있는가

<code>scopedToProfile()</code>	<code>schoolOnly</code> · <code>campusOnly</code> 로 남의 학교 공고를 목록·알림에서 제거
<code>notStale()</code>	마감일을 확정하지 못한 공고는 D-day가 없어 영영 안 사라진다 . 등록 60일 경과 시 숨김. 화면과 알림이 같은 함수를 쓴다 — 숨긴 공고를 알림으로 알리면 눌러도 찾을 수 없다
<code>requirementLines()</code> / <code>requirementMatch()</code>	구조화된 <code>eligibility</code> 가 없는 공고를 위해 원문 자격 문장 을 그대로 파싱해 대조 (소득분위·평점·특별자격 낱말·학년·재학 상태). 113건이 이 경로를 쓴다. "가./나."로 시작하는 줄을 버리면 진짜 요건이 날아가는 등, 실데이터에서 얻은 예외가 주석으로 남아 있다
<code>noticeForProfile()</code>	실시간 공고가 이 학생 것인지 판정. 본교/분교 게시판 공유 문제(한양 [서울]·[ERICA] 등)를 제목 표식으로 가른다

05 양식 엔진 — 이 제품의 기술적 해자

공고 첨부 HWP/PDF/DOCX 신청서를 **같은 구조의 문서로** 앱 안에서 만들어 준다. 종이 셋으로 갈라져 있고, 이 분리가 원칙 4("원본과 동일 구조")를 지키는 장치다.

① 스키마 (원본)

`data/forms.json` · 48종 · 필드 602개. 로봇 또는 사람이 원본 문서를 읽어 항목·문구·체크 보기를 그대로 옮긴 것.

단일 진실

② 질문 설계 (런타임)

`form-plan.js` — 스키마를 수정하지 않고 화면에 낼 질문 목록만 다시 짤다. 자동 채움·클릭형 승격·표 병합

③ 문서 생성

`forms.js renderFormDoc` —

①의 원본 스키마를 그대로

읽어 문서를 만든다. ②가 아무리 바뀌어도 문서는 안 바뀐다

질문 최적화 효과 — 원본 필드 602개가 학생에게는 질문 320개로

자동 채움 220

직접 입력 238

클릭 82

병합·머리칸 62

■ 프로필에서 자동으로 채움(22키) ■ 서술이 필요해 직접 입력 ■ 보기를 눌러 선택 ■ 표 병합·고정 머리칸

2026-08-18 작업 전후: 총 질문 675 → **320**, 직접입력 10개 초과 양식 25종 → **2종**, 전체 20개 초과 7종 → **0종**. 같은 기간 **48종 중 44종의** 생성 문서가 한 글자도 바뀌지 않았다(나머지 4종은 원본 대조로 데이터를 고친 것). 질문을 줄인 게 아니라 묻는 방식을 바꾼 것이라는 증거다.

레버	동작	효과
자동 채움	앱이 아는 값은 묻지 않는다. 로봇 생성 스키마에는 <code>auto</code> 표시가 0건이라 라벨을 정규화해 사전과 매칭한다(성명 / 성명 / 1. 성명 을 한 값으로 — 라벨 442종 → 408종)	220칸
클릭형 승격	보기가 데이터에 이미 적힌 필드만 <code>checks</code> → <code>choice</code> (단일 선택)로. 개수뿐 아니라 정확도 가 목적 — 그전엔 성별에 남·여를 둘 다 체크할 수 있었다	checks 69 중 62
표 병합	원본 표의 칸들을 카드 1개(<code>group</code>)로 묻고 문서에는 칸 수만큼 그대로 낸다. 직접입력이 10개를 넘을 때만 발동	조건부
고정 머리칸	표 머리 칸은 <code>static</code> 으로 묻지 않는다 — 원본 대조로 확인된 것만	6칸

불변식과 그 증명 장치

불변식: 질문 방식을 어떻게 바꾸든 `renderFormDoc`의 출력 HTML은 동일해야 한다. **증명:** `node verify/form-snapshot.mjs` — 48종의 생성 문서를 **문자 단위**로 기준본과 대조한다. 답을 채울 때 **필드 id가 아니라 라벨**로 채우는 것이 핵심이다(병합하면 id가 바뀌므로 id로 채우면 아무것도 증명하지 못한다).

상한: 직접입력 10 · 클릭 15 · 전체 20. **넘어도 질문을 감추지 않는다** — 감추면 학생이 빈 칸이 있는 문서를 제출하게 되므로, 전부 보여 주고 감사가 이름을 보고한다(현재 2종 초과).

06 데이터 스키마

data/registered.json — 정식 등록 공고 (188)

필드	보유	의미
id / name / provider	188	식별·주관
amount / amountValue	188 / 8	표시 문구 / 확정 금액(없으면 0)
deadline / period / listedAt	182 / 188 / 144	마감·기간·등록 일(staleness 기준)
eligibility{}	188	12신호 구조체
eligibilityLines[]	113	원문 자격 문장 그대로
excerpts / excerptNote	119	신청방법·서류 원문 발췌
formId	38	양식 연결(고유 36중)
noForm	144	양식 없음 사유 (감사 조건)
applyEmail	4	이메일 접수처
prepDoc	1	자유형식 확인 시에만 지원문서
auto	22	로봇 등록 = '검수 전' 배지
sourceUrl / attachments[]	188	원문·첨부 원본 링크

data/forms.json — 양식 스키마 (48)

```
{ title, docName, org, pledge, sections:[{ heading, info[][], fields[] }] }
```

info 는 인적사항 표(라벨↔프로필 키 쌍), fields 는 질문 항목이다. 한 섹션에 둘 다 있을 수 있다 — 예전에 info 가 있으면 fields 를 통째로 빠뜨리는 조기 return 버그가 있었다.

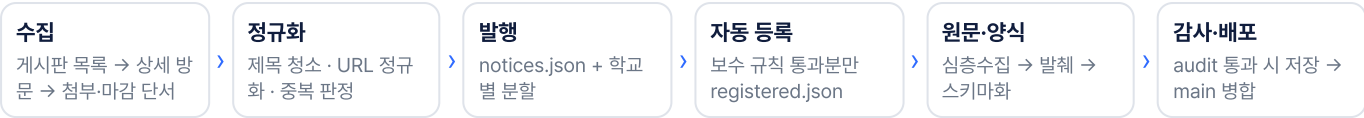
필드 타입	개수
text	412
textarea	102
checks	69
checks+text	8
static	6
schedule	4
choice	1

새 타입을 만들면 `verify/audit-data.js` 의 `FIELD_TYPES` 에 반드시 등록해야 한다 — 빠뜨리면 감사가 exit 1로 죽고 그날 수집 결과가 통째로 저장되지 않는다(워크플로가 되돌린다).

data/notices.json — 실시간 피드 (615)

제목·링크·학교·수집일·마감 단서·첨부만 담는다. **수집일 +60일에 자동 삭제**, 학교별 분할 파일(`data/notices/` 42개)로도 발행해 앱이 자기 학교 것만 받는다.

07 데이터 파이프라인 — 로봇 19기



모듈	하는 일 / 설계 이유
collect.mjs	일반 수집(HTTP). 게시판당 25건·학교당 30건 상한 — 상단 고정 공지가 10건 넘는 게시판에서 실공고가 잘리던 문제로 상향
browser-collect.mjs	진짜 Chromium. 봇 차단·동적 렌더링 게시판, 클릭형 (행이 링크가 아니라 스크립트인 경희대 유형)은 행을 실제로 눌러 상세를 채집. 인식 1건 이하면 진단(프레임 수·클릭 시도·본 글자·본 링크)이 리포트에 자동 첨부된다
auto-register.mjs	보수적 규칙(개별 실공고·미등록·마감 전·대출/행사/고용/대학원 제외)을 전부 통과한 것만 자동 등록. 애매한 건은 '컨펌 대기'로 리포트에만. 킬스위치 <code>enabled:false</code> , 오등록 제거 <code>blockIds</code>
deepfetch.mjs	공고 전문·첨부 원본을 <code>extracted/</code> 에 커밋 — 차단된 개발 샌드박스가 원문을 읽는 유일한 통로 . HWP는 <code>olefile</code> 로 <code>PrvText</code> 를, HWPX/DOCX는 zip 내부 XML을 풀어 텍스트까지 추출
extract-excerpts.mjs	확보한 전문에서 신청기간·방법·제출서류·문의 문장을 그대로 발체 → <code>excerpts</code> . 수집 때마다 자동 갱신
schema-from-text.mjs schematize-forms.mjs	무료 우선 분류기 : 글자가 깨끗한 원본은 규칙만으로(비용 0) 즉시 앱 양식으로 승격. PDF·표·시간표처럼 무료로 동일 문 서를 장담 못 하는 것만 Claude API 호출(실행당 상한 기본 2, 0 이면 완전 정지)
link-hunter.mjs	목록 주소로만 남은 공고 전담. 주소를 집착하지 않고 행을 눌러 브라우저가 간 주소를 받아 적는다. 재시도 간격 1→3→7→14→30일, 영구 포기 없음 . 3회째 <code>stuck</code> 이면 이슈 자동 생성
audit-coverage.mjs	주간 누락 감사 — 게시판에 있었는데 우리가 못 잡은 공고를 역으로 센다
build-search-index.mjs	도우미용 요약 색인. 원문 전문은 수 MB라 앱이 못 받으므로 그 공고에만 나오는 낱말만 남긴다(2건 이상 등장하면 버림 — 게시판 꺾데기 제거)

■ 예약 스케줄 (KST) — GitHub 예약 혼잡을 피해 전부 홀수 분

시각	워크플로	비고
05:13 (월)	원문 링크 복구	목록 주소로 남은 공고를 게시판에서 찾아 교정
06:23 (월)	공고 누락 감사	수집망 커버리지 역검증
06:37	링크 사냥꾼	본업은 사냥, 남는 시간에만 기존 링크 순찰
07:37	검색용 요약 만들기	도우미 색인 갱신
07:41 / 11:41	장학공고 수집	낮 실행은 아침이 생략됐을 때의 백업(중복은 <code>seen.json</code> 이 거른다)
08:07 / 12:07	브라우저형 수집	예산제 실행(<code>harvest-budget.mjs</code>) — 39분 상한
08:07 / 22:07	진척도 문서 갱신	PROGRESS.md 활동 로그 자동 기록
08:10 / 20:10	푸시 발송(Worker cron)	알릴 거리가 있는 학교의 구독자만 깨운다
12:23 / 16:23	배포 동기화	기본 브랜치 → main 병합. 수집 로봇 3종이 끝날 때마다도 실행
13:11	라이브 점검	실제 배포된 앱에 반영됐는지 확인
14:23	관리자 화면 잠금 확인	로그인 없이 열어 보고 잠김/안 잠김/판정 불가
14:37 / 22:37	main 직접 수정 되가져오기	배포 동기화의 반대 방향 짝

15:40 (월)

오래된 리포트 닫기

이슈 누적 방지

push-to-run 관행: MCP·수동 실행이 없는 세션에서도 `collector/run-*.txt` 파일을 고쳐 기본 브랜치에 push하면 해당 로봇이 즉시 돈다. 워크플로 저장 단계 규칙: `git add` 에 아직 없을 수 있는 파일을 넣으면 `fatal: pathspec` 으로 실행 전체가 실패한다 — 실제로 수집 8회가 연속 침몰한 사고가 있어, 선택 파일은 `[ -f ... ] && git add ...` 형태로만 넣고 실패 시  코멘트를 남기는 스텝(`if: failure()`)이 붙어 있다.

08 알림·푸시와 도우미 — 프라이버시 우선 설계

푸시 (server/push/worker.js ·가동 중)

서버가 하는 일은 "확인해 봐"라고 폰을 깨우는 것 하나다. 알림 문구는 서버가 만들지 않는다 — 깨어난 서비스워커가 기기 안의 프로필로 match-engine·notify-rules 를 돌려 스스로 문구를 만든다.

- 서버 저장 항목: 폰 주소 · 학교 · 캠퍼스뿐 (이름·성적·소득·서류는 애초에 전송하지 않음)
- 내용 없는 푸시라 암호화할 페이로드가 없다 → 서버 코드가 작고 고장날 곳이 적다
- 무료 등급 제약 2개를 구조로 회피: 실행당 외부 요청 50건 · CPU 10ms → reg → notices → plan → send (25대씩) 상태 머신으로 2분마다 한 걸음. 걸음이 죽어도 이어서 진행, 5회 실패 시 lastError 기록
- 404/410 응답 구독은 자동 삭제. /health · /test 로 실제 도달을 확인(KV의 등록 수는 '살아 있는 폰 수'가 아니다)

알림 5종: 새 매칭 공고 · 마감 D-1/당일 · 제출 리마인드 · 우리 학교 새 공고 · 결과 기록 리마인드. 한 번에 최대 12건(폭탄 방지), 첫 실행은 기존 공고를 '새 공고'로 쏟아내지 않도록 baseline 처리.

장학금 도우미 (chat.js + server/chat/)

답은 앱이 이미 아는 네 곳에서만 나온다: 등록 공고(엔진 판정 결과) · 실시간 피드(제목·링크만) · 내 신청 내역·보관함 · 공고 원문 발췌. 못 찾으면 지어내지 않고 "못 찾았다"고 말한다.

- 판정을 새로 만들지 않는다 — evaluate · dday 를 그대로 호출. 여기에 규칙을 한 벌 더 두면 화면과 챗봇이 다른 말을 한다
- AI는 기본 꺼져 있다(chat-config.js 의 endpoint가 비어 있음). 커도 역할은 '사서'로 한정 — 어느 공고의 몇 번째 인용문이 맞는지 고르기만 하고 답 문장을 쓰지 않는다(모델 claude-haiku-4-5, 항목 8·인용 6 상한)
- 전송 대상은 질문 + 앱이 고른 공고의 공개 정보뿐. 프로필은 보내지 않고 서버는 질문도 답도 저장하지 않는다(KV 없음)

09 검증 체계 — 이 프로젝트의 안전망

브라우저 자동화(Playwright) 드라이버 + 데이터 감사 도구로 24개. 커밋 전 실행이 관행이고, 일부는 워크플로 안에서 관문으로 돈다.

도구	검사 대상
audit-data.js	관문. 등록·양식 전수가 현재 엔진 기준을 충족하는지. 실패하면 그날 수집분을 저장하지 않고 되돌린다
form-snapshot.mjs	양식 48종의 생성 문서를 문자 단위로 대조 (원칙 4의 증명)
drive.js	전체 회귀 — 온보딩 → 매칭 → 보관함 → 신청 플로우
personas.js	120종 사용자 조합 스왑 — 크래시·빈 화면·콘솔 오류 탐지
verify-registered / new-forms / forms-data	등록 공고 노출, 신규 양식 렌더링, 데이터만 바뀌도 앱에 반영되는지
verify-source-links.js	'원문 공고 2'가 목록이 아니라 그 공고로 가는지 + 못 찾은 건 정직하게 표기되는지
verify-notify(-rules) / push-client / push-server	알림 규칙 단위 검증, 요청 본문 전수 검사로 개인정보 유출 여부, VAPID 서명 실검증
verify-chat.js	정직 답변·말귀(동의어·초성·오타)·가짜 AI 서버로 지어낸 답 차단·XSS
verify-admin.js	관리자 화면 — 열쇠 없이는 아무것도 안 보이는지, 등록 양식 전부가 오류 없이 문서를 만드는지
eligibility-report.mjs	자격 확보율 전수 재채점 — 규칙을 고치기 전후로 돌려 숫자를 비교하는 것이 이 영역의 작업 방식
check-deploy-sync / check-collab	작업본이 실제 앱에 나갔는지 / 상대 개발자와 같은 파일을 건드리는지

알려진 상태: verify-admin.js 4개 항목이 데이터 흐름 때문에 실패한다 — 마감 임박 공고가 1건뿐이라 '여러 건 선택' 검사가 성립하지 않는 것으로, 코드 결함이 아니며 공고가 늘면 저절로 통과한다.

10 브랜치·배포 토폴로지 (여기서 사고가 가장 많이 났다)

브랜치	역할
claude/nice-heisenberg-WESq5	기본 브랜치. GitHub의 예약 실행은 기본 브랜치 한 곳에서만 돌기 때문에 모든 로봇이 여기에 커밋한다
main	Pages 배포 소스. 여기 push해야 앱이 나간다
claude/work-seonju claude/work-josehyeon	개발자별 작업 브랜치. 두 사람 다 AI 세션으로 커밋해 작성자가 동일하게 찍히므로, 누구 작업인지는 브랜치로만 구분된다

두 방향 동기화 로봇

- `deploy-sync.yml`: 기본 브랜치 → main. **이게 없어서 수집 결과가 앱에 하루 이상 안 나간 사고**가 실제로 있었다
- `main-guard.yml`: main 직접 수정 → 기본 브랜치 되가져 오기
- 충돌 시 조용히 멈추지 않고 🚨 이슈를 만든다. 두 로봇은 서로 다른 **concurrency** 그룹이어야 한다(같은 그룹이면 GitHub이 대기분을 취소해 되가져오기가 사라진다)

● main에 브랜치 보호를 켜면 배포가 멈춘다

로봇이 main에 직접 **push**해서 앱을 내보내므로, "Require a pull request before merging"을 켜면 로봇이 함께 막혀 앱 업데이트가 통째로 멈춘다. 켤다면 **Block force pushes · Restrict deletions** 까지만. main 직접 수정 문제는 **막는 대신 고치는** 되가져오기 로봇으로 푼다.

■ 협업 충돌 해소 (COLLAB.md)

충돌의 대부분이 사람 코드가 아니라 **로봇이 매일 새로 쓰는 기록장**이었다(최근 100커밋 기준 notices 45·seen 36·리포트 28 ↔ `app.js` 5). 그래서 `.gitattributes` 로 전용 병합기(`tools/merge-json-union.mjs`)를 지정해 **합집합 병합**한다. 단 `registered.json`·`forms.json`은 **일부러 제외** — 여기서는 '삭제'가 의미를 가지므로(오등록 제거) 합집합이 지운 항목을 되살린다. `merge=ours`는 git 내장이 아니라 **클론마다 등록해야 동작한다**(`bash tools/setup-collab.sh`).

11 보안·프라이버시

영역	조치
XSS·데이터 오염	수집 데이터는 전부 신뢰할 수 없는 입력으로 취급. 모든 렌더링에 <code>esc()</code> , 링크는 <code>safeUrl()</code> 이 <code>javascript:·data:</code> 스킴을 <code>href.window.open</code> 에서 차단(CSP 위의 2차 방어)
CSP	<code>default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; connect-src 'self' https://*.workers.dev, referrer: no-referrer.</code> 즉 수집 데이터가 오염 돼도 코드 실행이 불가능하고 외부 전송 경로가 없다
개인정보	프로필·서류·답변은 기기 안(<code>localStorage/IndexedDB</code>)에만. 외부 fetch는 동일 오리진 <code>data/*.json</code> 과 푸시/챗 위 커뿐이며, 그 요청 본문에 프로필이 없다는 것을 <code>verify-push-client.js</code> 가 전수 검사한다
메일 서버	Origin 검사 + 수신 도메인 허용목록(<code>ac.kr/or.kr/go.kr/re.kr</code>) + 크기 제한 + 수신자 정규식 앵커링으로 스팸 릴레이 차단
관리자 콘솔	Cloudflare Access(허용 메일 2개·일회용 코드) + GitHub 토큰. 버튼이 저장소를 직접 고치지 않는다 — 워크플로를 깨우고 감사를 통과해야만 저장(실패 시 되돌림). 잠금은 매일 로봇이 로그인 없이 열어 보고 확인한다. 🗝️ 토큰은 2026-11-07 만료 — 화면이 알려 주면 재발급 필요
비밀값	API 키는 Cloudflare Secret / GitHub Secret에만. 워크플로 입력은 환경변수로만 셀에 전달

12 기술 부채와 90일 제안

순	항목	현황과 제안
1	테스트가 CI에 없다	드라이버는 훌륭한데 사람(세션)이 손으로 돌린다 . <code>audit-data</code> 만 워크플로 관문이다. → 앱 파일 변경 PR에 <code>drive·personas·form-snapshot</code> 을 붙이는 CI 1개 추가 (반나절)

2	수집 대상이 통제 밖	학교가 게시판을 바꾸면 그 학교만 조용히 0건. health·누락 감사·실패 이슈가 이미 있으나, 학교별 0건 연속 N일 경보 를 하나 더 세우는 것을 권함
3	자동 등록의 재고 압력	로봇이 매일 등록하므로 검수를 멈추면 '검수 전'이 쌓인다(현재 22건). 관리자 화면 일괄 검수가 있으니 주 1회 검수 리듬 을 운영 규칙으로 고정할 것
4	유료 경로 단절	Claude API 잔액 부족으로 8/11부터 PDF·표 서식 자동 스키마화가 정지. 무료 규칙 경로는 정상이라 앱은 멈추지 않지만 대기 23건이 밀린다
5	번들·타입 없음	의도적 선택이다(설치·빌드 0). 대신 전역 스크립트 순서와 <code>index.html · sw.js</code> ASSETS 배선이 수동이라 새 파일 추가 시 빠뜨리기 쉽다. → 배선 누락을 잡는 검사 1개면 충분, 번들러 도입은 권하지 않음
6	단일 저장소 결합도	앱·데이터·로봇·검증이 한 저장소에 있어 로봇 커밋이 사람 작업과 섞인다. 합집합 병합으로 완화돼 있고, 분리해 예약 실행 제약(기본 브랜치 한 곳) 때문에 득보다 실이 크다
7	회원가입·계정 없음	설계는 확정, 착수 보류. 기기 간 동기화가 없어 폰을 바꾸면 프로필이 사라진다. 착수 시 주민등록번호는 민감정보 별도 동의 대상 이라는 점이 설계 제약

CTO 첫 2주 추천 경로

① CLAUDE.md 의 **운영 원칙 8개**와 "**되돌리면 안 되는 지점**" 절부터 읽는다(이 저장소의 실질적 설계 문서). ② 로컬에서 `python3 -m http.server 8123 → verify/` 드라이버 전량 1회 실행으로 현재 기준선을 눈으로 확인. ③ `node verify/audit-data.js · eligibility-report.mjs` 로 데이터 품질 숫자를 잡는다. ④ 위 부채 1번(CI 관문)을 첫 커밋으로 — 이후 모든 작업의 안 전망이 된다.